Department of Computer Engineering

(Course NEP 2023)

LAB MANUAL

Year: TY B.Tech

Term: II

Course Name: Database Management System Lab

Course Code: 23UCOP3508

Index

Sr. No.	Contents
1	Institute Vision & Mission
2	Department Vision & Mission
3	List of PEO,PSO, POs
4	List of POs
5	Lab Manual

G H Raisoni College of Engineering & Management

VISION

To achieve excellent standards of quality education by keeping pace with rapidly changing technologies and create technical manpower of global standards with capabilities of accepting new challenges.

MISSION

Our efforts are dedicated to impart quality and value based education to raise satisfaction level of all stake-holders. Our strength is directed to create competent professionals. Our endeavor is to provide all possible support to promote research and development activities.

Department of Computer Engineering

VISION

To develop competent problem solvers and innovators with sound knowledge of Computer Engineering and urge to learn and effectively apply cutting edge technology .

MISSION

M 1: Imparting knowledge through innovative teaching pedagogies along with real life scenario based learning

M 2: Offering a solid platform to learn emerging technologies and applications and motivating for perusal of research

M 3: Inculcating the lifelong learning skills to empower to apply the most appropriate and effective technologies through outreach and continuous learning programs

Program Educational Objectives (PEOs)

PEO1: Ability to solve problems by applying the knowledge of Computer Engineering.

PEO2: Understand the interdisciplinary applications of computer engineering and provide the solutions to societal and industry problems

PEO3: Embed the values and morals to have appropriate use of rapidly changing computer technologies

PEO4: Get acquainted with the state of art and motivate for higher studies and research in computer engineering

Program Specific Outcomes (PSOs)

PSO1: Design and implement software solutions in core areas of Computing, Database Management, Networks and Web Technologies.

PSO2: Demonstrate skills in emerging areas addressing Security, Data Science, and the Internet of Things

Program Outcomes

PO 1: Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization for the solution of complex engineering problems.

PO 2: Problem analysis: Identify, formulate, research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO 3: Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet t h e specified needs with appropriate consideration for public health and safety, and cultural, societal, and environmental considerations.

PO 4: Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO 5: Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools, including prediction and modelling to complex engineering activities, with an understanding of the limitations.

PO 6: The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO 7: Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO 8: Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO 9: Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO 10: Communication: Communicate effectively on complex engineering activities with the engineering community and with t h e society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO 11: Project management and finance: Demonstrate knowledge and understanding of t h e engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO 12: Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Course Details

Course:

Class: T Y B.Tech. Division: A, B, C

Internal Marks: 25 **External marks: 25**

Credits : 1 Pattern: 2023

COURSE OUTCOME	
CO1:	Identify and summarize the fundamentals of relational database management systems and use them in database design.
CO2:	Describe Functional dependency and apply Structured Query Language (SQL) for database manipulation
CO3:	Implement transaction processing, concurrency control, and database recovery protocols in databases.
CO4:	Explore the use of NoSQL databases for processing unstructured data.

Sr. No.	Name of Experiments	CO Mapping
1.	Draw E-R diagram and convert entities and relationships to relation table for a given scenario. Two assignments shall be carried out i.e. consider two different scenarios (eg. bank, college, Employee, Hotel etc.)	**CO1**
	Consider the following database for Question (2 - 10) 1. **Online Banking System:** An online banking platform manages customers, their accounts, and transactions. **Key Tables:** • **Customers** (customer_id, name, email) • **Accounts** (account_id, customer_id, balance) • **Transactions** (transaction_id, account_id, amount, type, timestamp) 2. E-Commerce Inventory: An online store tracks customers, products, orders, and order items. **Key Tables:** • **Customers** (`customer_id`, `name`, `email`) • **Products** (`product_id`, `name`, `price`, `stock`) • **Orders** (`order_id`, `customer_id`, `order_date`) • **Order_Items** (`order_item_id`, `order_id`, `product_id`, `quantity`) 3. Hospital Management System: Manage patients, doctors, appointments, and treatments in a hospital. **Key Tables:** • **Patients** (`patient_id`, `name`, `age`, `gender`) • **Doctors** (`doctor_id`, `name`, `specialization`) • **Appointments** (`appointment_id`, `patient_id`, `doctor_id`, `date`) • **Treatments** (`treatment_id`, `appointment_id`, `diagnosis`, `prescription`) **4.** Library Management System**: A college library tracks books, students, borrow/return activities.** **5. Key Tables:** • **Books** (`book_id`, `title`, `author`, `copies_available`) • **Students** (`student_id`, `name`, `department`) • **Issues** (`issue_id`, `book_id`, `student_id`, `issue_date`, `return_date`)	

2	Implementation of DDL commands of SQL with suitable examples • Create table • Alter table • Drop Table	**CO2**
	To Perform the following: a. Viewing all databases, Viewing all Tables in a Database, Updating/Deleting Records in a Table	
4	Implementation of DML commands of SQL with suitable examples • Insert • Update • Delete.	**CO2**
5	For a given set of relation schemes, create tables and perform the following Simple Queries: Simple Queries with Aggregate functions, Queries with Aggregate functions (group by and having clause), Queries involving- Date Functions, String Functions , Math Functions	**CO2**
6	Implementation of different types of Joins • Inner Join • Outer Join • Natural Join etc.	**CO2**
7	To perform SQL query that demonstrate following: Search conditions, Summary queries, Sub- queries, Subqueries- With IN clause, With EXISTS clause	**CO2**
8	Perform Following • Creating Database /Table Space • Managing Users: Create User, Delete User • Managing roles:-Grant, Revoke.	**CO2**
9	Implementation of Database Backup & Recovery commands, Rollback, Commit, Savepoint.	**CO3**
10	Create View and index for database tables with a large number of records.	**CO3**
Open Ended Experiments / New Experiments		
1	Write PL/SQL Triggers for insert, delete, and update operations in a database table.	**CO4**
2	Database Connectivity: Write a program to implement MySQL/Oracle database connectivity with any front end language to implement Database navigation operations (add, delete, edit etc.)	

Experiment No. 1

Title

Draw E-R diagram and convert entities and relationships to relation tables for given scenarios.

Aim

To study the concept of Entity-Relationship (E-R) modeling and convert E-R diagrams into relational database tables for different real-world scenarios.

Theory

1. **Entity**
    - An object in the real world with an independent existence.
    - Example: *Student, Employee, Customer, Hotel, Room*.
    - Types:
        - Strong Entity → Exists independently.
        - Weak Entity → Depends on another entity.
2. **Attributes**
    - Properties or characteristics of an entity.
    - Example: *Student_Name, Roll_No, Employee_ID, Salary*.
    - Types:
        - Simple, Composite, Multivalued, Derived.
3. **Relationship**
    - Association between entities.
    - Example: *Student ENROLLED_IN Course, Customer BOOKS Room*.
    - Relationship Types:
        - One-to-One (1:1)
        - One-to-Many (1:M)
        - Many-to-Many (M:N)

4. **Keys**
    - Primary Key: Uniquely identifies a record.
    - Foreign Key: Attribute used to link entities.
5. **E-R Diagram**
    - A graphical representation of entities, attributes, and relationships.
    - Uses notations like rectangles (entities), ellipses (attributes), diamonds (relationships), lines (connections).
6. **Conversion of E-R Diagram to Relational Model**
    - Each strong entity → a table with its attributes.
    - Weak entity → a table including primary key of owner entity as foreign key.
    - 1:1 relationship → Merge or add foreign key.
    - 1:M relationship → Add foreign key on the "many" side.
    - M:N relationship → Create a new relation with foreign keys of both entities.

Procedure

1. Identify entities from the given scenario.
2. Find attributes of each entity and identify primary keys.
3. Identify relationships among entities.
4. Draw the E-R diagram.
5. Convert E-R diagram into relational tables.

Assignment – 1 (Scenario: College Database System)

Entities & Attributes

- **Student**(Student_ID, Name, Address, Phone, DOB)
- **Course**(Course_ID, Course_Name, Credits)
- **Faculty**(Faculty_ID, Name, Department, Phone)

Relationships

- Student **enrolls in** Course (M:N)
- Faculty **teaches** Course (1:M)

Conversion to Relational Tables

1. **Student**(Student_ID **PK**, Name, Address, Phone, DOB)
2. **Course**(Course_ID **PK**, Course_Name, Credits)
3. **Faculty**(Faculty_ID **PK**, Name, Department, Phone)
4. **Enrolls**(Student_ID **FK**, Course_ID **FK**, Enrollment_Date)
5. **Teaches**(Faculty_ID **FK**, Course_ID **FK**)

Assignment – 2 (Scenario: Hotel Management System)

Entities & Attributes

- **Customer**(Customer_ID, Name, Address, Phone)
- **Room**(Room_No, Type, Rent, Status)
- **Booking**(Booking_ID, Date_From, Date_To)

Relationships

- Customer **books** Room (M:N via Booking)
- Room **allocated to** Booking (1:M)

Conversion to Relational Tables

1. **Customer**(Customer_ID **PK**, Name, Address, Phone)
2. **Room**(Room_No **PK**, Type, Rent, Status)
3. **Booking**(Booking_ID **PK**, Customer_ID **FK**, Room_No **FK**, Date_From, Date_To)

Result

The E-R diagrams for **College** and **Hotel Management** systems were successfully drawn. Entities and relationships were identified and converted into relational database tables.

Viva Questions (for practical exam)

1. What is an entity? Differentiate between strong and weak entity.
2. What is the difference between primary key and foreign key?
3. Explain M:N relationship with example.
4. What are the steps to convert an E-R diagram into relational tables?

5. Why do we need a relational model after drawing E-R diagram?

Experiment No. 2

Title

Implementation of DDL commands of SQL with suitable examples.

Aim

To study and implement the Data Definition Language (DDL) commands in SQL such as **CREATE TABLE**, **ALTER TABLE**, and **DROP TABLE**.

Theory

1. **DDL (Data Definition Language)**
    - A subset of SQL commands used to define and modify the structure of database objects (tables, schemas, indexes, etc.).
    - Does not manipulate actual data; it only changes schema definitions.
2. **Important DDL Commands:**
    - **CREATE TABLE**
        - Used to create a new table in the database.
        - Syntax:

          CREATE TABLE table_name (

            column1 datatype constraints,

            column2 datatype constraints,

            ...

          );
    - **ALTER TABLE**
        - Used to modify an existing table (add, modify, or delete columns/constraints).
        - Syntax:

          ALTER TABLE table_name ADD column_name datatype;

          ALTER TABLE table_name MODIFY column_name datatype;

ALTER TABLE table_name DROP column_name;

- **DROP TABLE**
  - Used to delete a table permanently from the database (removes structure and data).
  - Syntax:

    DROP TABLE table_name;

Procedure

1. Start the SQL database environment (e.g., MySQL, Oracle, PostgreSQL, etc.).
2. Use **CREATE TABLE** to define a new table with attributes.
3. Apply **ALTER TABLE** commands to modify the structure of the table.
4. Finally, use **DROP TABLE** to remove the table permanently.

Example

(Scenario: Online Banking System)

1. **CREATE TABLE**

   CREATE TABLE Customers (

   customer_id INT PRIMARY KEY,

   name VARCHAR(50),

   email VARCHAR(50)

   );
2. **ALTER TABLE – Add a new column**

   ALTER TABLE Customers ADD phone VARCHAR(15);
3. **ALTER TABLE – Modify column**

   ALTER TABLE Customers MODIFY name VARCHAR(100);
4. **ALTER TABLE – Drop column**

   ALTER TABLE Customers DROP COLUMN email;
5. **DROP TABLE**

   DROP TABLE Customers;

Result

The DDL commands **CREATE TABLE, ALTER TABLE, and DROP TABLE** were successfully implemented in SQL with suitable examples.

Viva Questions

1. What is the difference between DDL and DML in SQL?
2. Can we recover a table after using the DROP TABLE command?
3. What is the difference between DROP and TRUNCATE?
4. Why do we use ALTER TABLE instead of recreating a table?
5. Can we add constraints using ALTER TABLE? Give an example.

Experiment No. 3

Title

To perform the following operations in SQL:
a) Viewing all databases
b) Viewing all tables in a database
c) Updating and Deleting records in a table

Aim

To learn how to view databases, list tables in a database, and perform update/delete operations on records using SQL commands.

Theory

1. **Viewing all Databases**
    - Helps identify the available databases in the database management system.
    - Syntax (MySQL):

      SHOW DATABASES;
2. **Viewing all Tables in a Database**
    - Once inside a database, we can view all available tables.
    - Syntax:

      USE database_name;

      SHOW TABLES;
3. **Updating Records in a Table**
    - Used to modify existing records in a table.
    - Syntax:

      UPDATE table_name

      SET column_name = new_value

      WHERE condition;
4. **Deleting Records in a Table**
    - Used to remove records from a table permanently.
    - Syntax:

DELETE FROM table_name

WHERE condition;

If the WHERE clause is omitted, all records in the table will be deleted.

Procedure

1. Open the SQL environment (MySQL/Oracle/PostgreSQL).
2. View all databases using the SHOW DATABASES command.
3. Select a particular database using USE.
4. List all tables inside the database using SHOW TABLES.
5. Use UPDATE to modify specific records.
6. Use DELETE to remove unwanted records.

Example (Using E-Commerce Inventory Database)

1. Viewing all Databases

   SHOW DATABASES;

2. Selecting and Viewing Tables in a Database

   USE Ecommerce;

   SHOW TABLES;

3. Updating a Record in Customers Table

   UPDATE Customers

   SET email = 'john.doe@gmail.com'

   WHERE customer_id = 101;

4. Deleting a Record from Products Table

   DELETE FROM Products

   WHERE product_id = 205;

Result

We successfully executed SQL queries to view databases, list tables, and update/delete records in tables.

Viva Questions

1. What is the difference between DROP TABLE and DELETE FROM?
2. What happens if we use DELETE FROM table_name without a WHERE clause?
3. Can UPDATE modify multiple records at once? Explain with an example.
4. Why is it important to use the WHERE clause in update and delete queries?
5. Which SQL command is used to view all the databases present in MySQL?

Experiment No. 4

Title

Implementation of DML commands of SQL with suitable examples (Insert, Update, Delete).

Aim

To study and implement the Data Manipulation Language (DML) commands in SQL such as **INSERT**, **UPDATE**, and **DELETE** with suitable examples.

Theory

1. **DML (Data Manipulation Language)**
    - DML is a subset of SQL used to **manage and manipulate the data** stored in database tables.
    - Unlike **DDL (Data Definition Language)** which deals with schema (structure), DML commands deal with actual **records (rows)** in tables.
    - DML is most frequently used in day-to-day operations by database users and administrators.
2. **Types of DML Commands**

a) INSERT Command

- Used to add new records into a table.
- Can insert a single record or multiple records at once.
- Syntax:
- INSERT INTO table_name (column1, column2, column3, ...)
- VALUES (value1, value2, value3, ...);
- If values for all columns are provided, column names can be omitted:
- INSERT INTO table_name VALUES (value1, value2, value3, ...);

b) UPDATE Command

- Used to modify existing records in a table.
- Can update one or more columns at a time.
- Syntax:
- UPDATE table_name

- SET column1 = value1, column2 = value2, ...
- WHERE condition;

Without a WHERE clause, **all rows** in the table will be updated.

c) DELETE Command

- Used to remove records from a table.
- Syntax:
- DELETE FROM table_name WHERE condition;
- If no condition is provided, all records will be deleted.

3. **Difference between DML and DDL**

Feature	DDL (Data Definition Language)	DML (Data Manipulation Language)
Purpose	Defines/changes schema	Manipulates actual data
Examples	CREATE, ALTER, DROP	INSERT, UPDATE, DELETE, SELECT
Effect	Structure level	Data level

Procedure

1. Start the SQL database environment.
2. Create or select a database.
3. Create a table for performing operations (if not already created).
4. Use **INSERT** to add new records into the table.
5. Use **UPDATE** to modify certain attributes of existing records.
6. Use **DELETE** to remove unwanted records from the table.

Example (Using Hospital Management System)

1. **Creating a Table (for Patients)**

```
CREATE TABLE Patients (
    patient_id INT PRIMARY KEY,
    name VARCHAR(50),
```

```
    age INT,
    gender VARCHAR(10)
);
```

2. **INSERT Command**

```
INSERT INTO Patients (patient_id, name, age, gender)
VALUES (101, 'Ramesh Kumar', 45, 'Male');

INSERT INTO Patients VALUES (102, 'Priya Sharma', 30, 'Female');
```

3. **UPDATE Command**

```
UPDATE Patients
SET age = 46
WHERE patient_id = 101;
```

4. **DELETE Command**

```
DELETE FROM Patients
WHERE patient_id = 102;
```

Result

The DML commands **INSERT, UPDATE, and DELETE** were successfully implemented in SQL with suitable examples.

Viva Questions

1. What is the difference between DDL and DML commands?
2. What happens if we use UPDATE without a WHERE clause?
3. Differentiate between DELETE and TRUNCATE.
4. Can we insert multiple rows with a single INSERT statement? Give an example.
5. Why is the WHERE clause important in UPDATE and DELETE queries?

Experiment No. 5

Title

For a given set of relation schemes, create tables and perform the following queries:

- Simple Queries with Aggregate Functions
- Queries with Aggregate Functions (GROUP BY and HAVING clause)
- Queries involving Date Functions, String Functions, and Math Functions

---

Aim

To study and implement different types of SQL queries using aggregate functions, group operations, and inbuilt SQL functions such as date, string, and math functions.

Theory

1. **Aggregate Functions**
    - Aggregate functions perform calculations on a set of values and return a single value.
    - Common Aggregate Functions:
        - **COUNT()** → Returns number of rows.
        - **SUM()** → Returns total of numeric column.
        - **AVG()** → Returns average value.
        - **MAX()** → Returns maximum value.
        - **MIN()** → Returns minimum value.
    - Example:
    - SELECT COUNT(*) FROM Transactions;
    - SELECT AVG(balance) FROM Accounts;
2. **GROUP BY Clause**
    - Used to arrange identical data into groups.
    - Example:
    - SELECT customer_id, COUNT(account_id)
    - FROM Accounts
    - GROUP BY customer_id;

3. **HAVING Clause**
    - Used to filter groups (works like WHERE but for groups).
    - Example:
    - SELECT customer_id, COUNT(account_id)
    - FROM Accounts
    - GROUP BY customer_id
    - HAVING COUNT(account_id) > 1;
4. **Date Functions**
    - Functions to handle and manipulate date values.
    - Examples:
        - **CURDATE()** → Returns current date.
        - **YEAR(date)** → Extracts year.
        - **MONTH(date)** → Extracts month.
        - **DATEDIFF(date1, date2)** → Difference between two dates.
5. **String Functions**
    - Used to manipulate text data.
    - Examples:
        - **LENGTH(string)** → Returns length.
        - **UPPER(string)** → Converts to uppercase.
        - **LOWER(string)** → Converts to lowercase.
        - **SUBSTRING(string, start, length)** → Extracts part of string.
        - **CONCAT(str1, str2, …)** → Joins strings.
6. **Math Functions**
    - Built-in functions to perform mathematical operations.
    - Examples:
        - **ABS(x)** → Absolute value.
        - **ROUND(x, d)** → Round value to d decimal places.
        - **CEIL(x)** → Smallest integer greater than or equal to x.
        - **FLOOR(x)** → Largest integer less than or equal to x.

- **POWER(x, y)** → x raised to the power y.

Procedure

1. Create the required tables in the database.
2. Insert sample records into the tables.
3. Execute queries using aggregate functions (SUM, AVG, MAX, MIN, COUNT).
4. Apply GROUP BY and HAVING clauses on grouped data.
5. Write queries using Date, String, and Math functions.
6. Verify results of each query.

Example (Online Banking System)

Step 1: Create Tables

```
CREATE TABLE Customers (
   customer_id INT PRIMARY KEY,
   name VARCHAR(50),
   email VARCHAR(50)
);

CREATE TABLE Accounts (
   account_id INT PRIMARY KEY,
   customer_id INT,
   balance DECIMAL(10,2),
   FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)
);

CREATE TABLE Transactions (
   transaction_id INT PRIMARY KEY,
   account_id INT,
   amount DECIMAL(10,2),
```

```
    type VARCHAR(10),
    timestamp DATE,
    FOREIGN KEY (account_id) REFERENCES Accounts(account_id)
);
```

Step 2: Insert Sample Data

```
INSERT INTO Customers VALUES (1, 'Ramesh Kumar', 'ramesh@gmail.com');
INSERT INTO Customers VALUES (2, 'Priya Sharma', 'priya@yahoo.com');

INSERT INTO Accounts VALUES (101, 1, 50000);
INSERT INTO Accounts VALUES (102, 1, 25000);
INSERT INTO Accounts VALUES (103, 2, 30000);

INSERT INTO Transactions VALUES (1001, 101, 5000, 'Credit', '2025-09-01');
INSERT INTO Transactions VALUES (1002, 101, 2000, 'Debit',  '2025-09-05');
INSERT INTO Transactions VALUES (1003, 102, 7000, 'Credit', '2025-09-10');
INSERT INTO Transactions VALUES (1004, 103, 1500, 'Debit',  '2025-09-12');
```

Queries and Examples

a) Simple Queries with Aggregate Functions

```
SELECT COUNT(*) FROM Customers;            -- Number of customers
SELECT SUM(balance) FROM Accounts;         -- Total balance in bank
SELECT AVG(balance) FROM Accounts;         -- Average account balance
SELECT MAX(amount) FROM Transactions;      -- Maximum transaction amount
SELECT MIN(amount) FROM Transactions;      -- Minimum transaction amount
```

b) Queries with GROUP BY and HAVING Clause

```
-- Total number of accounts per customer
SELECT customer_id, COUNT(account_id) AS total_accounts
FROM Accounts
GROUP BY customer_id;
```

```
-- Customers with more than 1 account
SELECT customer_id, COUNT(account_id) AS total_accounts
FROM Accounts
GROUP BY customer_id
HAVING COUNT(account_id) > 1;


-- Total transaction amount grouped by type
SELECT type, SUM(amount) AS total
FROM Transactions
GROUP BY type;
```

c) Queries involving Date Functions

```
SELECT CURDATE();                          -- Current date
SELECT YEAR(timestamp), COUNT(*)
FROM Transactions
GROUP BY YEAR(timestamp);                  -- Transactions per year
SELECT DATEDIFF(CURDATE(), timestamp)
FROM Transactions;                         -- Days since each transaction
```

d) Queries involving String Functions

```
SELECT UPPER(name) FROM Customers;              -- Customer names in uppercase
SELECT SUBSTRING(name, 1, 5) FROM Customers;    -- First 5 letters of names
SELECT CONCAT(name, ' - ', email) FROM Customers;   -- Combine name and email
```

e) Queries involving Math Functions

```
SELECT ABS(-2500);                         -- Absolute value
SELECT ROUND(123.456, 2);                  -- Rounds to 2 decimals
SELECT CEIL(45.12), FLOOR(45.12);          -- Ceil and Floor
SELECT POWER(2, 5);                        -- 2^5 = 32
```

Result

The given queries using aggregate functions, grouping with HAVING, and built-in date, string, and math functions were executed successfully on the Online Banking System database.

Viva Questions

1. What is the difference between WHERE and HAVING clause?
2. Can we use aggregate functions without GROUP BY? Give an example.
3. Name any three string functions in SQL and their use.
4. What is the difference between COUNT(*) and COUNT(column_name)?
5. Give examples of SQL math functions and their purpose.

Experiment No. 6

Title

Implementation of different types of Joins (Inner Join, Outer Join, Natural Join, etc.)

Aim

To study and implement different types of **SQL Joins** such as **INNER JOIN**, **OUTER JOIN**, **NATURAL JOIN**, and others with suitable examples.

Theory

1. **Joins in SQL**
    - Joins are used to **combine data from two or more tables** based on related columns.
    - A join condition is usually based on a **foreign key – primary key relationship**.
2. **Types of Joins**

a) INNER JOIN

- Returns only the rows that have matching values in both tables.
- Rows with no match are excluded.
- Syntax:

```
SELECT columns
FROM table1
INNER JOIN table2
ON table1.column = table2.column;
```

b) LEFT OUTER JOIN (LEFT JOIN)

- Returns all rows from the **left table**, and the matched rows from the right table.
- Non-matching rows in the right table are shown as NULL.
- Syntax:

```
SELECT columns
FROM table1
```

LEFT JOIN table2

ON table1.column = table2.column;

c) RIGHT OUTER JOIN (RIGHT JOIN)

- Returns all rows from the **right table**, and the matched rows from the left table.
- Syntax:

  SELECT columns

  FROM table1

  RIGHT JOIN table2

  ON table1.column = table2.column;

d) FULL OUTER JOIN

- Returns all rows from both tables.
- Where there is no match, NULL is placed in the missing side.
- Syntax (in some SQL engines like PostgreSQL):

  SELECT columns

  FROM table1

  FULL OUTER JOIN table2

  ON table1.column = table2.column;

e) NATURAL JOIN

- Joins two tables automatically based on columns with the **same name** and compatible data types.
- No need to specify ON condition explicitly.
- Syntax:

  SELECT columns

  FROM table1

  NATURAL JOIN table2;

f) CROSS JOIN (Cartesian Join)

- Produces the Cartesian product of two tables (every row of table1 with every row of table2).
- Syntax:

```
SELECT columns
FROM table1
CROSS JOIN table2;
```

3. **Difference Between Inner and Outer Joins**

Feature	Inner Join	Outer Join (Left/Right/Full)
Matching Rows	Only rows with matches	Includes unmatched rows as well
NULL Handling	No NULLs introduced	NULLs added for missing matches
Usage	Strict filtering	Ensures no data loss from one side

Procedure

1. Create the required tables in the database.
2. Insert sample data into the tables.
3. Apply **INNER JOIN** queries between related tables.
4. Apply **LEFT JOIN, RIGHT JOIN, and FULL OUTER JOIN** to see unmatched records.
5. Use **NATURAL JOIN** and **CROSS JOIN** to explore automatic and Cartesian joins.
6. Observe results and verify correctness.

Example (E-Commerce Inventory Database)

Step 1: Create Tables

```
CREATE TABLE Customers (
   customer_id INT PRIMARY KEY,
   name VARCHAR(50),
   email VARCHAR(50)
);
CREATE TABLE Orders (
   order_id INT PRIMARY KEY,
   customer_id INT,
   order_date DATE,
```

```
    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)
);
```

Step 2: Insert Sample Data

```
INSERT INTO Customers VALUES (1, 'Ramesh Kumar', 'ramesh@gmail.com');
INSERT INTO Customers VALUES (2, 'Priya Sharma', 'priya@yahoo.com');
INSERT INTO Customers VALUES (3, 'Amit Verma', 'amit@gmail.com');

INSERT INTO Orders VALUES (101, 1, '2025-09-01');
INSERT INTO Orders VALUES (102, 1, '2025-09-02');
INSERT INTO Orders VALUES (103, 2, '2025-09-03');
```

Queries

a) INNER JOIN

```
SELECT Customers.name, Orders.order_id, Orders.order_date
FROM Customers
INNER JOIN Orders
ON Customers.customer_id = Orders.customer_id;
```

Returns only customers who placed orders.

---

b) LEFT OUTER JOIN

```
SELECT Customers.name, Orders.order_id
FROM Customers
LEFT JOIN Orders
ON Customers.customer_id = Orders.customer_id;
```

Returns all customers, including those with no orders (NULL for order_id).

---

c) RIGHT OUTER JOIN

```
SELECT Customers.name, Orders.order_id
FROM Customers
```

```
RIGHT JOIN Orders
ON Customers.customer_id = Orders.customer_id;
```

Returns all orders, even if they have no matching customer record.

---

d) FULL OUTER JOIN *(if supported)*

```
SELECT Customers.name, Orders.order_id
FROM Customers
FULL OUTER JOIN Orders
ON Customers.customer_id = Orders.customer_id;
```

Returns all customers and all orders, matching where possible.

---

e) NATURAL JOIN

```
SELECT *
FROM Customers
NATURAL JOIN Orders;
```

Automatically joins on customer_id column.

---

f) CROSS JOIN

```
SELECT Customers.name, Orders.order_id
FROM Customers
CROSS JOIN Orders;
```

Returns all possible combinations of customers and orders.

Result

Different types of joins were successfully implemented on the E-Commerce Inventory database, showing how data from multiple tables can be combined and analyzed.

Viva Questions

1. What is the difference between INNER JOIN and LEFT OUTER JOIN?
2. Can we use NATURAL JOIN if column names differ in the two tables?
3. What is the result of a CROSS JOIN between two tables of sizes m and n?
4. Explain with an example how HAVING can be used with a JOIN.
5. Why is JOIN preferred over subqueries in many cases?

Experiment No. 7

Title

To perform SQL queries that demonstrate:

- Search conditions
- Summary queries
- Sub-queries
- Subqueries with IN clause
- Subqueries with EXISTS clause

Aim

To learn and implement different types of SQL queries including **search conditions, summary queries, and subqueries** (with IN and EXISTS clauses) for retrieving meaningful data from relational databases.

Theory

1. **Search Conditions**
    - Search conditions are used in the WHERE clause to filter records.
    - They may involve comparison operators (=, <, >, !=), logical operators (AND, OR, NOT), and pattern matching (LIKE).
    - Example:

      SELECT * FROM Students WHERE age > 20 AND department = 'CSE';
2. **Summary Queries (Aggregate Queries)**
    - These are queries that perform calculations on a set of values and return a single value.
    - SQL provides aggregate functions like:
        - COUNT() → number of rows
        - SUM() → total value
        - AVG() → average value
        - MAX() → maximum value
        - MIN() → minimum value

- Example:

  SELECT department, COUNT(*) AS total_students

  FROM Students

  GROUP BY department;

3. **Subqueries**
   - A **subquery** is a query within another query.
   - Types:
     - **Single-row subquery** → returns one value
     - **Multi-row subquery** → returns multiple values
     - **Correlated subquery** → executes once for each row in the outer query
4. **Subquery with IN Clause**
   - Used when the subquery returns **multiple values**.
   - Example:

     SELECT name FROM Students

     WHERE student_id IN (SELECT student_id FROM Enrollments WHERE course_id = 'CSE101');
5. **Subquery with EXISTS Clause**
   - Used to test whether a subquery returns any rows.
   - Returns TRUE if the subquery has at least one record.
   - Example:

     SELECT name FROM Students s

     WHERE EXISTS (SELECT * FROM Enrollments e WHERE s.student_id = e.student_id);

Procedure

1. Create the required tables (Students, Courses, Enrollments).
2. Insert sample records.
3. Perform queries using **search conditions**.
4. Execute **summary queries** with aggregate functions.

5. Write **subqueries** with IN and EXISTS.
6. Verify results and record observations.

Example (University Database)

Step 1: Create Tables

```
CREATE TABLE Students (
   student_id INT PRIMARY KEY,
   name VARCHAR(50),
   age INT,
   department VARCHAR(30)
);

CREATE TABLE Courses (
   course_id VARCHAR(10) PRIMARY KEY,
   course_name VARCHAR(50),
   credits INT
);

CREATE TABLE Enrollments (
   enrollment_id INT PRIMARY KEY,
   student_id INT,
   course_id VARCHAR(10),
   FOREIGN KEY (student_id) REFERENCES Students(student_id),
   FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);
```

Step 2: Insert Sample Data

```
INSERT INTO Students VALUES (1, 'Ravi Sharma', 21, 'CSE');
INSERT INTO Students VALUES (2, 'Priya Mehta', 20, 'ECE');
INSERT INTO Students VALUES (3, 'Amit Gupta', 22, 'CSE');
```

```
INSERT INTO Students VALUES (4, 'Neha Rani', 19, 'ME');

INSERT INTO Courses VALUES ('CSE101', 'Database Systems', 4);
INSERT INTO Courses VALUES ('ECE201', 'Digital Circuits', 3);

INSERT INTO Enrollments VALUES (101, 1, 'CSE101');
INSERT INTO Enrollments VALUES (102, 3, 'CSE101');
INSERT INTO Enrollments VALUES (103, 2, 'ECE201');
```

Queries and Examples

a) Search Conditions

```
SELECT name, department
FROM Students
WHERE age > 20 AND department = 'CSE';
```

Returns CSE students older than 20.

---

b) Summary Query (Aggregate Functions)

```
SELECT department, COUNT(*) AS total_students
FROM Students
GROUP BY department;
```

Counts students in each department.

---

c) Simple Subquery

```
SELECT name
FROM Students
WHERE student_id = (SELECT student_id FROM Enrollments WHERE
enrollment_id = 101);
```

Returns student enrolled with enrollment_id = 101.

---

d) Subquery with IN Clause

```
SELECT name
FROM Students
WHERE student_id IN (SELECT student_id FROM Enrollments WHERE course_id
= 'CSE101');
```

Returns all students enrolled in Database Systems.

---

e) Subquery with EXISTS Clause

```
SELECT name
FROM Students s
WHERE EXISTS (SELECT * FROM Enrollments e WHERE s.student_id =
e.student_id);
```

Returns all students who are enrolled in at least one course.

Result

The SQL queries were successfully executed to demonstrate **search conditions, summary queries, subqueries, and subqueries with IN and EXISTS clauses**.

Viva Questions

1. What is the difference between IN and EXISTS in subqueries?
2. Can a subquery return multiple columns? Explain with an example.
3. What is the difference between a correlated subquery and a nested subquery?
4. Why do we use GROUP BY in summary queries?
5. Write a query to find students not enrolled in any course.

Experiment No. 8

Title

To perform the following database administration tasks:

- Creating Database / Table Space
- Managing Users (Create User, Delete User)
- Managing Roles (Grant, Revoke)

Aim

To understand and implement the concepts of **database creation, user management, and role management** in SQL by performing operations like creating a database, creating/deleting users, and granting/revoking privileges.

Theory

1. **Creating Database / Tablespace**
    - A **database** is a collection of related data stored in an organized manner.
    - In Oracle, the logical storage unit is called a **tablespace**, which holds tables, indexes, and other objects.
    - In MySQL or PostgreSQL, creating a database automatically creates a default tablespace.
    - **Syntax (MySQL/PostgreSQL):**

      CREATE DATABASE database_name;
2. **Managing Users**
    - A **user** is an account created in the DBMS to allow individuals to log in and access data.
    - **Creating a User**:

      CREATE USER username IDENTIFIED BY password;
    - **Dropping a User**:

      DROP USER username;
    - Users must be granted **privileges** before they can perform actions on databases/tables.
3. **Managing Roles**

- A **role** is a collection of privileges that can be granted to users.
- **Grant Command**: Used to assign privileges to users/roles.

  GRANT privilege_name ON object_name TO username;
- **Revoke Command**: Used to take back privileges from users/roles.

  REVOKE privilege_name ON object_name FROM username;

4. **Privileges**
   - **System Privileges**: Creating sessions, databases, tables, etc.
   - **Object Privileges**: SELECT, INSERT, UPDATE, DELETE on tables or views.

Procedure

1. Start the SQL environment (MySQL/Oracle/SQL Server).
2. Create a new **database** or **tablespace**.
3. Create a user with a password.
4. Grant privileges to the user to access the database.
5. Verify privileges by performing operations with the new user.
6. Revoke privileges and delete the user.

Example (MySQL)

Step 1: Create Database

```
CREATE DATABASE CollegeDB;
```

Step 2: Create User

```
CREATE USER 'ravi'@'localhost' IDENTIFIED BY 'ravi123';
```

Step 3: Grant Privileges to User

```
GRANT SELECT, INSERT, UPDATE ON CollegeDB.* TO 'ravi'@'localhost';
```

Step 4: Revoke Privileges from User

```
REVOKE INSERT, UPDATE ON CollegeDB.* FROM 'ravi'@'localhost';
```

Step 5: Delete User

```
DROP USER 'ravi'@'localhost';
```

Result

The operations for creating a database/tablespace, managing users, and managing roles (granting/revoking privileges) were successfully executed.

Viva Questions

1. What is the difference between a database and a tablespace?
2. Why do we create roles in SQL instead of granting privileges to each user directly?
3. Differentiate between **system privileges** and **object privileges**.
4. What is the use of the CASCADE option while dropping a user in Oracle?
5. Can a user exist without any privileges? What will happen if they try to log in?

Experiment No. 9

Title

Implementation of Database Backup & Recovery commands, Rollback, Commit, and Savepoint.

Aim

To study and implement the commands used for **backup & recovery of databases**, and to understand **transaction control commands** such as ROLLBACK, COMMIT, and SAVEPOINT in SQL.

Theory

1. **Database Backup and Recovery**
    - **Backup** is the process of creating a copy of the database to safeguard against data loss.
    - **Recovery** is the process of restoring data from a backup in case of corruption, hardware failure, or accidental deletion.
    - Types of Backup:
        - **Full Backup** → Entire database.
        - **Incremental Backup** → Only changes since the last backup.
        - **Logical Backup** → Exporting schema/data (e.g., mysqldump, Oracle expdp).
        - **Physical Backup** → Copying database files directly.
    - Example (MySQL Command Line):

      mysqldump -u root -p CollegeDB > CollegeDB_backup.sql;

      Recovery:

      mysql -u root -p CollegeDB < CollegeDB_backup.sql;
2. **Transaction Control Language (TCL) Commands**
   Transactions ensure that a group of SQL statements executes as a single unit of work.
    - **COMMIT**
        - Saves all changes made during the transaction permanently in the database.
        - Syntax:

COMMIT;

- **ROLLBACK**
  - Undoes changes made in the current transaction before they are committed.
  - Syntax:

    ROLLBACK;
- **SAVEPOINT**
  - Sets a point within a transaction to which you can roll back.
  - Allows partial undo of a transaction.
  - Syntax:

    SAVEPOINT savepoint_name;

    ROLLBACK TO savepoint_name;

3. **Importance of Backup, Recovery, and TCL**
   - Backup & recovery ensure **data availability** even after failures.
   - COMMIT ensures data consistency.
   - ROLLBACK helps undo mistakes.
   - SAVEPOINT gives finer control over transactions.

Procedure

1. Start SQL server and create a database.
2. Perform backup using database export tools/commands.
3. Drop or modify data to simulate accidental deletion.
4. Restore database using recovery command.
5. Create a table and perform transaction operations:
   - Insert data without commit.
   - Use rollback to undo.
   - Use savepoints to roll back partially.
   - Finally commit to save permanently.

Example (Using MySQL/Oracle)

Step 1: Create Table

```
CREATE TABLE Accounts (
   account_id INT PRIMARY KEY,
   holder_name VARCHAR(50),
   balance DECIMAL(10,2)
);
```

Step 2: Insert Records (Start Transaction)

```
START TRANSACTION;

INSERT INTO Accounts VALUES (101, 'Ravi Kumar', 5000);
INSERT INTO Accounts VALUES (102, 'Neha Sharma', 7000);
```

Step 3: Set a Savepoint

```
SAVEPOINT sp1;

INSERT INTO Accounts VALUES (103, 'Amit Gupta', 9000);
```

Step 4: Rollback to Savepoint

```
ROLLBACK TO sp1;
```

Row for Amit will be undone, but Ravi & Neha remain.

Step 5: Commit the Transaction

```
COMMIT;
```

Changes for Ravi & Neha are now permanent.

Step 6: Rollback Entire Transaction (Example)

```
START TRANSACTION;

UPDATE Accounts SET balance = balance - 1000 WHERE account_id = 101;

ROLLBACK; -- Undo the deduction
```

Result

The commands for **backup, recovery, commit, rollback, and savepoint** were executed successfully, demonstrating how data can be secured, restored, and transactions managed effectively.

Viva Questions

1. What is the difference between logical and physical backup?
2. Explain the difference between ROLLBACK and ROLLBACK TO SAVEPOINT.
3. What happens if we forget to use COMMIT after an insert statement?
4. Can we roll back after a commit? Why or why not?
5. Why are backup and recovery critical in enterprise systems?

Experiment No. 10

Title

Create View and Index for database tables with a large number of records.

Aim

To study and implement the creation of **views** and **indexes** in SQL for database tables containing a large number of records, and to understand their importance in query simplification and performance optimization.

Theory

1. **View in SQL**
    - A **view** is a **virtual table** based on the result of a query.
    - It does not store data itself but displays data stored in other tables.
    - Benefits of Views:
        - Simplifies complex queries.
        - Provides a level of security by restricting access to specific columns.
        - Allows customized representation of data.
    - Syntax:

      CREATE VIEW view_name AS

      SELECT column1, column2

      FROM table_name

      WHERE condition;
    - Example:

      CREATE VIEW CSE_Students AS

      SELECT student_id, name

      FROM Students

      WHERE department = 'CSE';
    - Types of Views:
        - **Simple View** → Based on one table.
        - **Complex View** → Based on multiple tables (joins, group functions).

        - **Materialized View** (Oracle) → Stores actual data for faster access.

2. **Index in SQL**
    - An **index** is a database object created on a table column to speed up data retrieval.
    - Works like an index in a book → makes searches faster.
    - **Advantages**:
        - Improves performance of SELECT queries.
        - Especially useful for large tables.
    - **Disadvantages**:
        - Increases storage requirements.
        - Slows down INSERT, UPDATE, and DELETE (because index must also be updated).
    - Syntax:

      CREATE INDEX index_name

      ON table_name (column_name);
    - Example:

      CREATE INDEX idx_student_dept

      ON Students(department);
3. **When to Use Views and Indexes**
    - **Views** → for simplifying queries, restricting access, or presenting summarized data.
    - **Indexes** → for improving query speed when searching or filtering large datasets.

Procedure

1. Create a database and tables with large records.
2. Insert sample records into tables.
3. Create a **view** to simplify a query.
4. Create an **index** to improve performance of queries on frequently used columns.
5. Test queries before and after indexing.

Example (University Database)

Step 1: Create Table (with many records)

```
CREATE TABLE Students (
   student_id INT PRIMARY KEY,
   name VARCHAR(50),
   age INT,
   department VARCHAR(30)
);
```

Step 2: Insert Sample Records

```
INSERT INTO Students VALUES (1, 'Ravi Sharma', 21, 'CSE');
INSERT INTO Students VALUES (2, 'Priya Mehta', 20, 'ECE');
INSERT INTO Students VALUES (3, 'Amit Gupta', 22, 'CSE');
INSERT INTO Students VALUES (4, 'Neha Rani', 19, 'ME');
-- Assume thousands of rows are inserted here...
```

Step 3: Create a View

```
CREATE VIEW CSE_Students AS
SELECT student_id, name
FROM Students
WHERE department = 'CSE';
```

Simplifies query to fetch only CSE students.

Step 4: Create an Index

```
CREATE INDEX idx_dept
ON Students(department);
```

Queries filtering by department will run faster.

Step 5: Using the View

```
SELECT * FROM CSE_Students;
```

Result

The **view** and **index** were successfully created, making queries simpler and faster for large datasets.

Viva Questions

1. What is the difference between a **table** and a **view**?
2. Can we update data through a view? Explain.
3. What are the disadvantages of creating too many indexes?
4. Differentiate between **simple view** and **complex view**.
5. What is the difference between **clustered index** and **non-clustered index**?